

Lecture 21: Randomness

*Lecturer: Abraham Ladha**Scribe(s): Diksha Holla*

Randomness is a funny thing. The topics of today's lecture is algorithms which might be really hard to solve without some creativity

1 Driving

First algorithm, remembering how to drive. There I was going 17 mph on a 40 mph road in the suburbs when I forgot which foot goes on which pedal. I was approaching a car stopped at a light and had to think fast. I had the choice between the following two algorithms:

algorithm 1:

look in the glove box for the manual and
find the page on which pedal does what.

algorithm 2:

just guess.

Algorithm 1 requires what, logn? n steps? In a constrained scenario I can't afford that. Choosing alg 1 may seem like the only idea but in the time constraint there is failure (and death) with probability 1. The second alg seems ridiculous but it only takes constant time! It has a failure rate of only 1/2. I just guessed which pedal was gas and which was brake and stopped just in time. The power of randomness!

2 Primality

Lets talk about a more realistic problem, primality testing. On input n, determine if n is prime or not. There are algorithms that tests all factors up to $\sqrt{n}$. Can we do better? Turns out with the power of randomness we can!

Before seeing the next algorithm, try to come up with your own deterministic primality testing algorithm.

Recall Fermats little theorem: $\text{prime} \Rightarrow a^{p-1} \equiv 1 \pmod{p}$ we can use this to test primality of a general n.

```
def f(n):  
    a = math.random(z_1, ... n-z)  
    if a^(n-1) = 1 (mod n)  
        return true  
    else  
        return false
```

The arithmetic can be done faster using repeated squaring but this at least takes poly-time. If n is prime it always returns true, but if n is not prime it doesn't always return false. There do exist numbers which may pass this test and not be prime. Carmichael numbers. They are rare but they do exist. Only if less than 10,000. Odds you hit one are negligible but never zero.

Okay randomness did enable this faster primality testing algorithm. was there not a fast deterministic primality testing algorithm? It took humanity until 2004!! to find one. [AKS04] uses derandomization techniques to give a deterministic polytime primality testing algorithm. we have known about primality for millennia, and took us this long to find this algorithm. It's the case that AKS04 is too advanced to present today, but just know it exists and was an achievement.

What about algorithms for comparing pi? $\pi = 3.14...$ This is actually surprisingly hard. Everyone just uses `math.pi` or something, and assume its stored to sufficient precision. But if you had to compute it, how would you?

Recall $\tan(\frac{\pi}{4}) = 1$, so compute $\pi = 4\tan^{-1}(1) = 4(1 - 1/3 + 1/5 - 1/7 + 1/9 - \dots)$ by rules of the Taylor series, we can compute $4\tan^{-1}(1)$. The problem with this is each term takes approx. the same or more operations to compute, but the amount we get to π gets smaller and smaller. For G digits of precision, we need like a billion operations :(

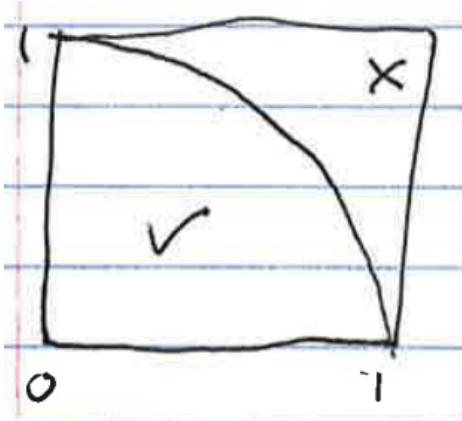
Theres a better way. Just guess! Throw a fistfull of darts at the board, whats the probability that it hits? Its proportional to the area of the circle. The probability a dart hits is the area!



$$P[\text{dart hits circle}] = \text{darts hit} / \text{total darts} \approx \frac{\pi r^2}{4r^2} \approx \pi/4$$

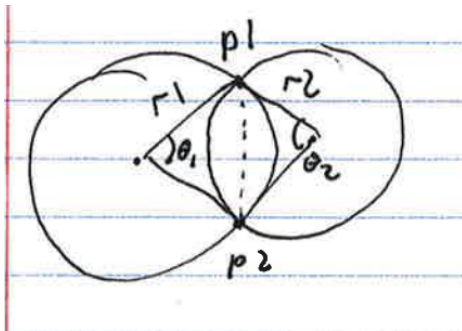
so we can guess π by throwing darts! lets optimize a bit

$$P[\text{darts hit circle}] = \text{darts hit} / \text{total darts} = \frac{\pi r^2/4}{r^2} \approx \pi/4$$

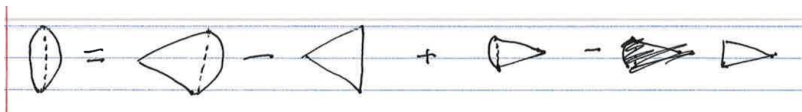


a constant speed up to compute the same value. How do you determine if the dart hits the circle though? if $d = (x, y)$, then $x^2 + y^2 = r^2$ is the boundary. For the unit circle and sphere, if $x^2 + y^2 \leq 1$, then its a hit, $x^2 + y^2 > 1$ is a miss. as we throw more and more darts, the error rate of our algorithm from approx. π will converge. It will do so slowly, yet faster than compiling Taylor series that's for sure.

Okay I admit its not faster than computing the Taylor series, and its less accurate, but it generalizes in ways the Taylor series solution doesn't.



Here's a classical geometry way. let circles 1, 2 intersect at points p1, p2. Then we have true triangles of lengths r1, r1, γ r2, r2, x where γ is the distance between p1 and p2. there are circle arcs as pie pieces. This gives us a way to compute the sliver, the almond.



steps:

1. find p1, p2 from the radii and centers
2. find distance between p1, p2, called x
3. let $t1 = \text{area}(r1, r1, x)$
4. let $t2 = \text{area}(r2, r2, x)$
5. use c1r1, c2r2 to find θ_1 and θ_2
6. sector 1 = $\theta_1 2r_1^2$

7. sector 2 = $\theta_2 2r_2^2$
8. return s1 - t1 ts2 - t2

Or, using the method of Monte-Carlo, just guess the area! a little math must be done to determine the bounding box, but that's not too bad and an overestimate won't hurt. This method works for this problem really since the question only asks for the digits of precision.

One final algorithm. Given a graph G , 2-color the graph so at least half the edges have endpoints with different colors.

Guess each node to be either red or blue. Then each edge has a $1/2$ chance to be different or same endpoint. If $\geq E/2$ edges have different endpoints, halt. else, recolor. Because endpoints of each are chosen randomly, and independently, the chance our first colors is valid is very high. The probability we have successive coloring's without the property should converge exponentially.